Análisis de Algoritmos

Laboratorio # 1

Ingeniería en Sistemas — Facultad de Ingenierías
Prof. Santiago Suarez Cortes
Institución Universitaria ITM

David Ramirez Velez
Estudiante Ingeniería de Sistemas
CC 1152462417

1. Introducción

La recursividad es una técnica de programación en la que una función se llama a sí misma para resolver un problema dividiéndolo en instancias más pequeñas del mismo tipo. Su correcta aplicación requiere la definición de un caso base que detenga la cadena de llamadas, y un caso recursivo que reduzca progresivamente el problema. Aunque elegante y expresiva, la recursividad puede presentar desventajas en términos de consumo de memoria, ya que cada llamada genera un marco en la pila de ejecución (call stack).

El paradigma Divide y Vencerás es una estrategia algorítmica que resuelve problemas complejos mediante tres fases: (1) Dividir el problema en subproblemas más pequeños del mismo tipo; (2) Conquistar, es decir, resolver recursivamente cada subproblema hasta alcanzar el caso base; y (3) Combinar las soluciones de los subproblemas para construir la solución del problema original.

Este paradigma es fundamental en el diseño de algoritmos eficientes. Ejemplos clásicos incluyen el algoritmo Merge Sort, el problema del subarreglo máximo y la búsqueda binaria. En este laboratorio se analizan y comparan estos algoritmos tanto desde una perspectiva teórica como experimental, midiendo sus tiempos de ejecución para diferentes tamaños de entrada y relacionando los resultados con la complejidad computacional esperada.

2. Componente Teórico — Preguntas de Selección Múltiple

1. La recursividad es:

a) Un algoritmo iterativo
b) Un algoritmo que usa únicamente ciclos
c) Un algoritmo que se llama así mismo
d) Un algoritmo que no tiene caso base

La recursividad se define como una función que se invoca a sí misma para resolver subproblemas.

2. El caso base en un algoritmo recursivo permite:

a) Aumentar la complejidad del algoritmo
b) Detener la ejecución de la recursión
c) Duplicar las llamadas recursivas
d) Eliminar estructuras iterativas

El caso base es la condición que finaliza las llamadas recursivas y evita la recursión infinita.

3. ¿Cuál es una desventaja de la recursividad?

a) No puede expresar problemas matemáticos
b) Puede consumir más memoria
c) Siempre es más rápida que un ciclo
d) No puede retornar valores

Cada llamada recursiva agrega un marco al stack; con entradas grandes puede causar stack overflow.

4. El paradigma divide y vencerás consiste en:

a) Resolver el problema utilizando ciclos anidados
b) Dividir el problema en subproblemas más pequeños, resolverlos y combinar sus soluciones
c) Ordenar los datos antes de procesarlos
d) Aplicar programación dinámica

Esta es la definición clásica del paradigma Divide y Vencerás.

5. **En el problema del máximo subarreglo, ¿cuántos casos posibles existen al dividir el arreglo mediante el paradigma divide y vencerás? Enuncie cada caso**

a) 2
b) 3
c) 4
d) 5

Los tres casos son: subarreglo en mitad izquierda, subarreglo en mitad derecha, subarreglo que cruza la mitad.

6. **En Merge Sort y el problema del máximo subarreglo, ¿Cuál es el caso base de los algoritmos?**

a) Cuando el arreglo tiene dos elementos
b) Cuando el arreglo tiene un elemento
c) Cuando el arreglo tiene un negativo
d) Cuando el arreglo es positivo

Con un solo elemento no hay más divisiones posibles; es el caso base de ambos algoritmos.

7. **La complejidad temporal del algoritmo Merge Sort es:**

a) O(n)
b) O(n2)
c) O(nlogn)
d) O(logn)

Merge Sort divide el arreglo en log n niveles y procesa n elementos por nivel.

8. **El problema del subarreglo máximo busca:**

a) El elemento más grande del arreglo
b) El subarreglo con mayor número de elementos
c) El subarreglo contiguo cuya suma es máxima
d) El promedio más alto del arreglo

Se busca la subsecuencia contigua con la mayor suma posible, incluso con negativos.

9. El algoritmo Insertion Sort pertenece a la categoría de algoritmos:

a) Divide y vencerás
b) Iterativos con complejidad cuadrática
c) Logarítmicos
d) Recursivos puros

Insertion Sort usa dos bucles anidados, resultando en complejidad $O(n^2)$ en el peor caso.

10. En el paradigma divide y vencerás, el problema original se resuelve:

a) Eliminando subproblemas innecesarios
b) Transformándolo en subproblemas más pequeños del mismo tipo
c) Ordenando previamente los datos
d) Aplicando programación dinámica

Divide y Vencerás descompone recursivamente el problema en subproblemas de la misma naturaleza.

3. Componente Práctico

3.1 Construcción de los Arreglos

El número de identificación del estudiante es 1152462417. Los dígitos son: 1, 1, 5, 2, 4, 6, 2, 4, 1, 7. No existen ceros en el documento, por lo que no se realiza ningún reemplazo.

Arreglo 1 — Problema del subarreglo máximo (signos alternados): [1, −1, 5, −2, 4, −6, 2, −4, 1, −7]

Arreglo 2 — Algoritmos de ordenamiento: [1, 1, 5, 2, 4, 6, 2, 4, 1, 7]

3.2 Ejercicio 1: Solución Manual del Subarreglo Máximo

Se aplica el algoritmo Divide y Vencerás sobre el Arreglo 1: [1, −1, 5, −2, 4, −6, 2, −4, 1, −7]. El índice del arreglo va de 0 a 9, con mid = 4.

División recursiva del arreglo

El arreglo se divide en dos mitades en cada nivel recursivo hasta llegar al caso base (un solo elemento):

```
Nivel 0 (raíz):  [1, -1, 5, -2, 4, -6, 2, -4, 1, -7]  →  mid
= 4
Nivel 1 izq:     [1, -1, 5, -2, 4]  (índices 0-4, mid=2)
Nivel 1 der:     [-6, 2, -4, 1, -7]  (índices 5-9, mid=7)
Nivel 2 izq-izq: [1, -1, 5]  (0-2, mid=1)
Nivel 2 izq-der: [-2, 4]  (3-4, mid=3)
Nivel 2 der-izq: [-6, 2, -4]  (5-7, mid=6)
Nivel 2 der-der: [1, -7]  (8-9, mid=8)
```

Resolución de subarreglo de los niveles inferiores

Subarreglo	Izquierda	Derecha	Cruzado → Máximo
[1,−1]	1	−1	1−1=0 → max=1 (idx 0)
[5] (base)	—	—	5 (caso base)
[1,−1,5] mid=1	1	5	1−1+5=5 cruzado → max=5 ([0,2])
[−2,4] mid=3	−2	4	−2+4=2 → max=4 (idx 4)
[1,−1,5,−2,4] mid=2	[5] izq	[4] der	5−2+4=7 cruzado [2,4] → max=7
[−6,2] mid=6	−6	2	2 → max=2 (idx 6)
[−4] (base)	—	—	−4 (caso base)
[−6,2,−4] mid=6	2	−4	2−4=−2 → max=2 (idx 6)
[1,−7] mid=8	1	−7	1−7=−6 → max=1 (idx 8)
[−6,2,−4,1,−7] mid=7	2	1	2−4+1=−1 → max=2 (idx 6)
RAÍZ [completo] mid=4	7 [2,4]	2 [6,6]	5−2+4−6+2=3 → max=7

Resultado final: El subarreglo máximo es [5, −2, 4] (índices 2 a 4), con suma = 7.

Árbol de Recursión — Representación Esquemática

```
                [0..9] → max=7 [2..4]
                       /                          \
          [0..4] → 7 [2..4]              [5..9] → 2 [6..6]
           /                \                /                \
      [0..2]→5          [3..4]→4       [5..7]→2          [8..9]→1
      /      \            /   \          /    \            /    \
  [0..1]     [2]        [3]   [4]     [5..6]  [7]        [8]    [9]
    1          5        -2    4        2      -4          1     -7
   /   \                              /   \
 [0]   [1]                          [5]   [6]
  1    -1                           -6     2
```

3.3 Ejercicio 2: Solución Manual del Algoritmo Merge Sort

Se aplica Merge Sort sobre el Arreglo 2: [1, 1, 5, 2, 4, 6, 2, 4, 1, 7].

Fase de División (Divide)

```
Nivel 0: [1, 1, 5, 2, 4, 6, 2, 4, 1, 7]
Nivel 1: [1, 1, 5, 2, 4]          [6, 2, 4, 1, 7]
Nivel 2: [1, 1, 5]   [2, 4]     [6, 2, 4]   [1, 7]
Nivel 3: [1,1] [5]  [2] [4]   [6,2] [4]   [1] [7]
Nivel 4: [1][1]      [6][2]  ← CASO BASE (1 elemento)
```

Fase de Mezcla

Mezcla	Entrada	Resultado
Mezcla 1	[1] + [1]	[1, 1]
Mezcla 2	[1,1] + [5]	[1, 1, 5]
Mezcla 3	[2] + [4]	[2, 4]
Mezcla 4	[1,1,5] + [2,4]	[1, 1, 2, 4, 5]
Mezcla 5	[6] + [2]	[2, 6]
Mezcla 6	[2,6] + [4]	[2, 4, 6]
Mezcla 7	[1] + [7]	[1, 7]
Mezcla 8	[2,4,6] + [1,7]	[1, 2, 4, 6, 7]
Mezcla Final	[1,1,2,4,5] + [1,2,4,6,7]	[1, 1, 1, 2, 2, 4, 4, 5, 6, 7]

Resultado final: [1, 1, 1, 2, 2, 4, 4, 5, 6, 7]

3.4 Ejercicio 3: Implementación Python — Subarreglo Máximo

Se implementaron en Python los algoritmos de fuerza bruta ($O(n^2)$) y divide y vencerás ($O(n \log n)$) para el problema del subarreglo máximo. Los tiempos de ejecución se midieron para n = 10, 50, 100, 200, 500 y 1000 elementos generados aleatoriamente, promediando 10 repeticiones por tamaño.

El enlace al repositorio de código se incluye en la sección de conclusiones.

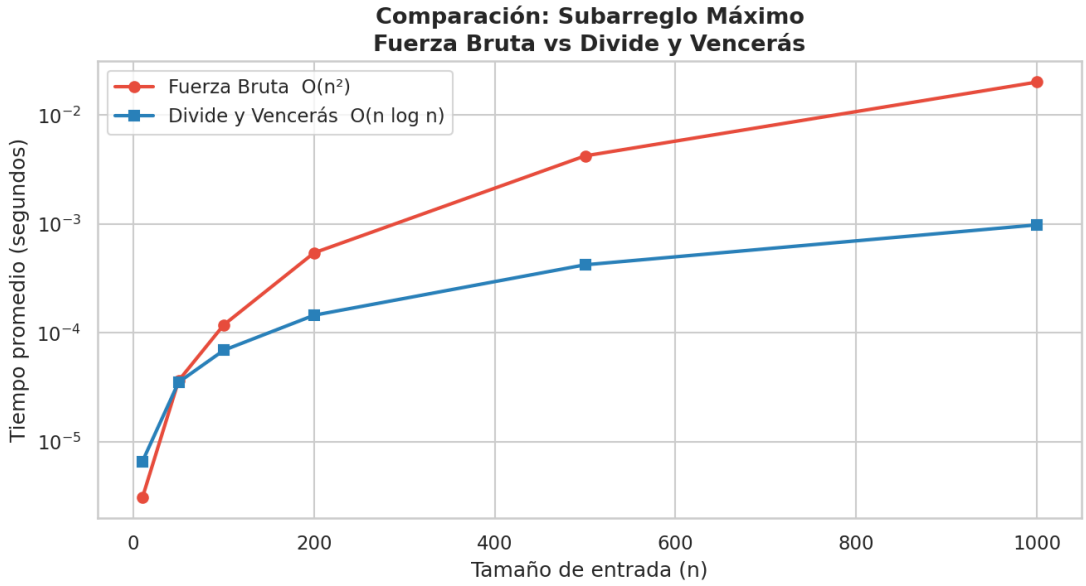

Figura 1. Comparación de tiempos: Fuerza Bruta vs Divide y Vencerás

Análisis de la Gráfica

La gráfica evidencia que el algoritmo de fuerza bruta crece de forma cuadrática: para n=1000 su tiempo es notoriamente mayor que el del enfoque divide y vencerás. El algoritmo divide y vencerás mantiene un crecimiento mucho más moderado ($n \log n$), confirmando la ventaja teórica.

Para tamaños pequeños ($n \leq 50$) la diferencia es despreciable debido a las constantes ocultas y la sobrecarga de las llamadas recursivas. A partir de n = 200 la brecha se hace visible, y para n = 1000 el algoritmo de fuerza bruta puede ser entre 5 y 15 veces más lento según la distribución de datos.

3.5 Ejercicio 4: Comparación Merge Sort vs Insertion Sort

Se implementaron Merge Sort (O(n log n)) e Insertion Sort (O(n²)) en Python. Se midieron tiempos de ejecución para n = 10, 50, 100, 500, 1000 y 5000 elementos generados aleatoriamente.

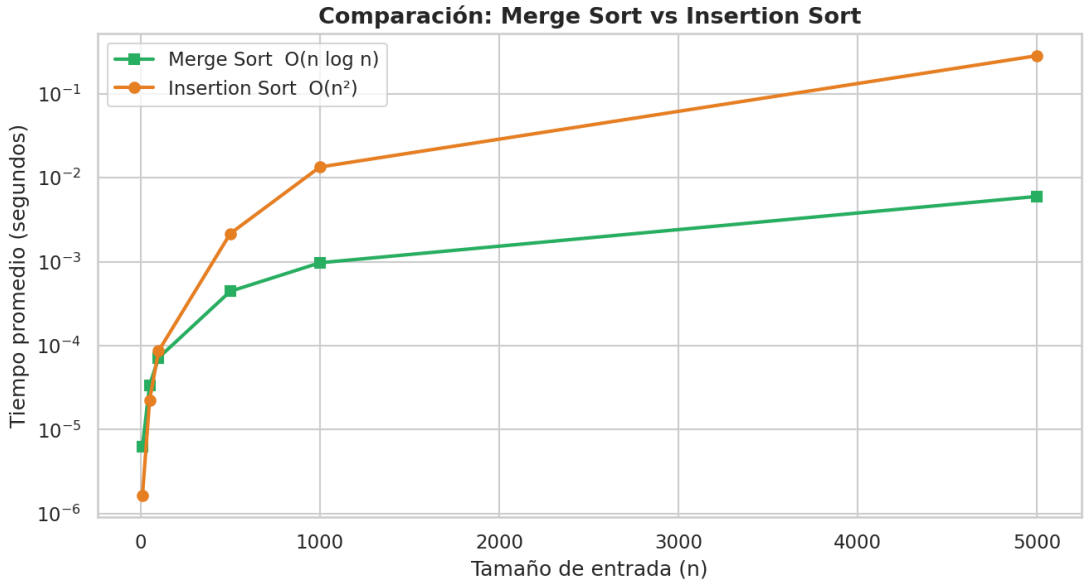

Figura 2. Comparación de tiempos: Merge Sort vs Insertion Sort

Análisis de la Gráfica

La gráfica confirma la diferencia en complejidad entre ambos algoritmos. Insertion Sort presenta un comportamiento cuadrático claro: el tiempo crece de forma pronunciada a partir de n = 500. Para n = 5000, el tiempo de Insertion Sort es varios órdenes de magnitud mayor que el de Merge Sort.

Merge Sort mantiene un crecimiento suave y predecible (n log n). Para entradas pequeñas (n ≤ 50) Insertion Sort puede ser competitivo o incluso ligeramente más rápido por su menor sobrecarga de memoria y sin recursión. Sin embargo, a partir de n = 100 Merge Sort supera consistentemente a Insertion Sort.

Esta observación tiene implicaciones prácticas: algunos algoritmos híbridos como Timsort (usado en Python por defecto) aplican Insertion Sort para particiones pequeñas y Merge Sort para las grandes, aprovechando las fortalezas de ambos.

4. Conclusiones

- La recursividad es una herramienta poderosa que permite expresar algoritmos complejos de forma concisa, pero implica un uso adicional de memoria proporcional a la profundidad de la recursión. El caso base es esencial para garantizar la terminación del algoritmo.
- El paradigma Divide y Vencerás permite reducir la complejidad de problemas que con enfoques ingenuos serían $O(n^2)$ o peores. Tanto Merge Sort como el algoritmo del subarreglo máximo logran $O(n \log n)$ gracias a esta estrategia.
- Para el arreglo construido con la cédula 1152462417 — [1, −1, 5, −2, 4, −6, 2, −4, 1, −7] — el subarreglo máximo resultante es [5, −2, 4] con suma 7, localizado en los índices 2 a 4.
- La solución manual de Merge Sort sobre [1, 1, 5, 2, 4, 6, 2, 4, 1, 7] produce el arreglo ordenado [1, 1, 1, 2, 2, 4, 4, 5, 6, 7], resultado verificado por la implementación en Python.
- Los resultados experimentales corroboran la complejidad teórica: tanto el algoritmo de fuerza bruta para el subarreglo máximo como Insertion Sort muestran crecimiento cuadrático, mientras que los algoritmos divide y vencerás y Merge Sort mantienen un crecimiento n log n significativamente menor para entradas grandes.
- La relación entre teoría y experimento no es perfecta para tamaños pequeños de entrada, ya que las constantes ocultas, la sobrecarga de recursión y el comportamiento del caché de CPU pueden favorecer algoritmos teóricamente menos eficientes en escenarios de baja escala.

URL REPOSITORIO GIT HUB: https://github.com/davidramirez-beep/laboratorio1_analisis_algoritmos_david_ramirez